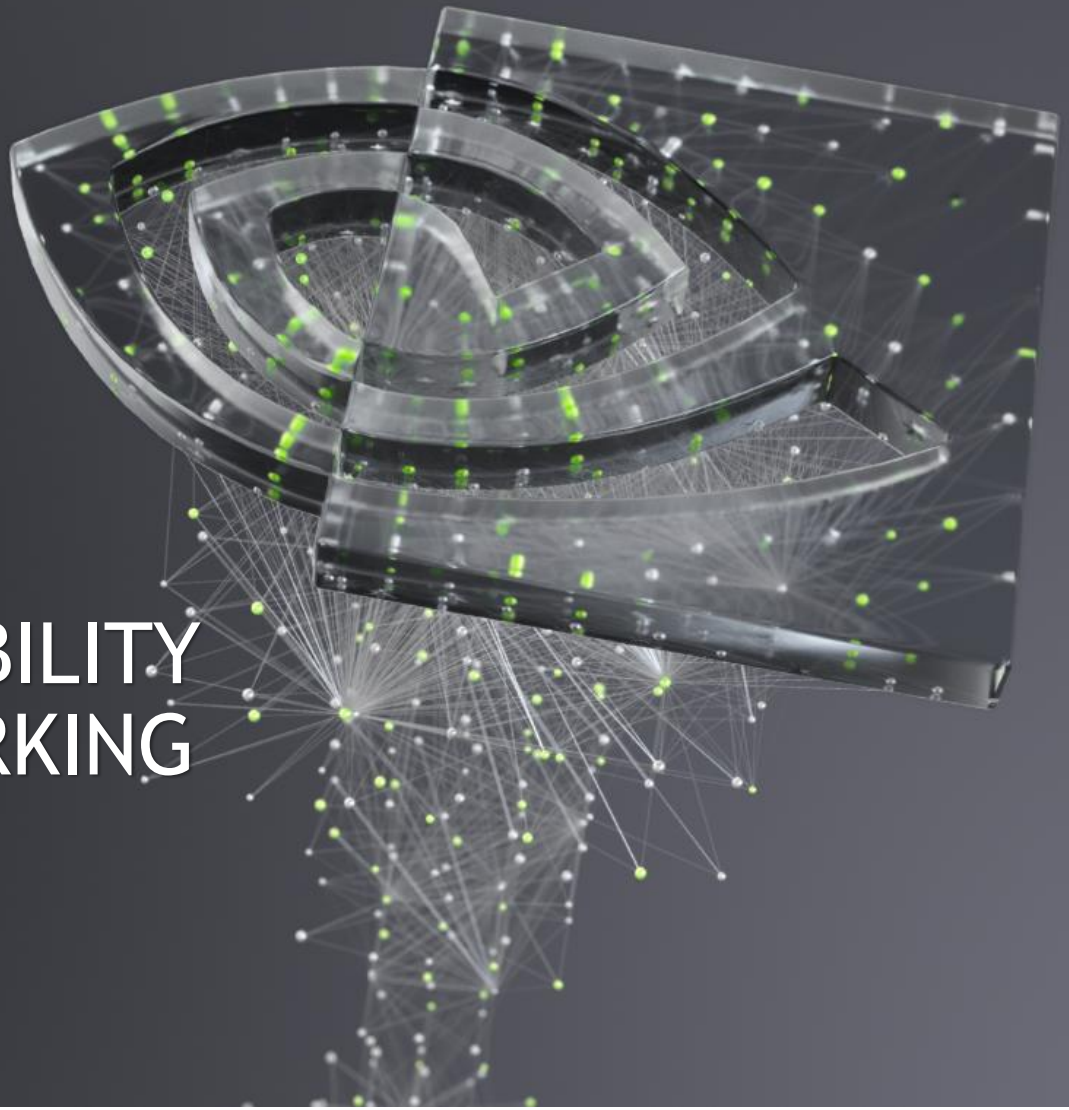


NETWORK WIDE VISIBILITY WITH LINUX NETWORKING AND SFLOW

July 2021

AGENDA – FIRST PART

- Switch ASICs in Linux
- Improvements in switch ASICs observability in Linux
  - Packet sampling
  - Packet drops monitoring

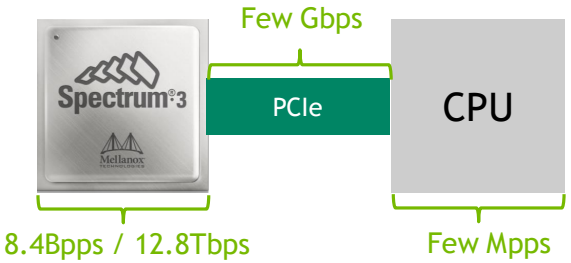
NVIDIA.

SWITCH ASICS IN LINUX



SWITCH ASICS IN LINUX

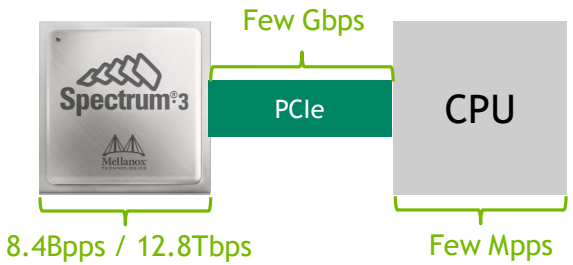




NVIDIA.

SWITCH ASICS IN LINUX

- Packets are forwarded between front panel ports by the switch ASIC
- Invisible to the host CPU
- Severe degradation in observability compared to a CPU-based solution



NVIDIA.

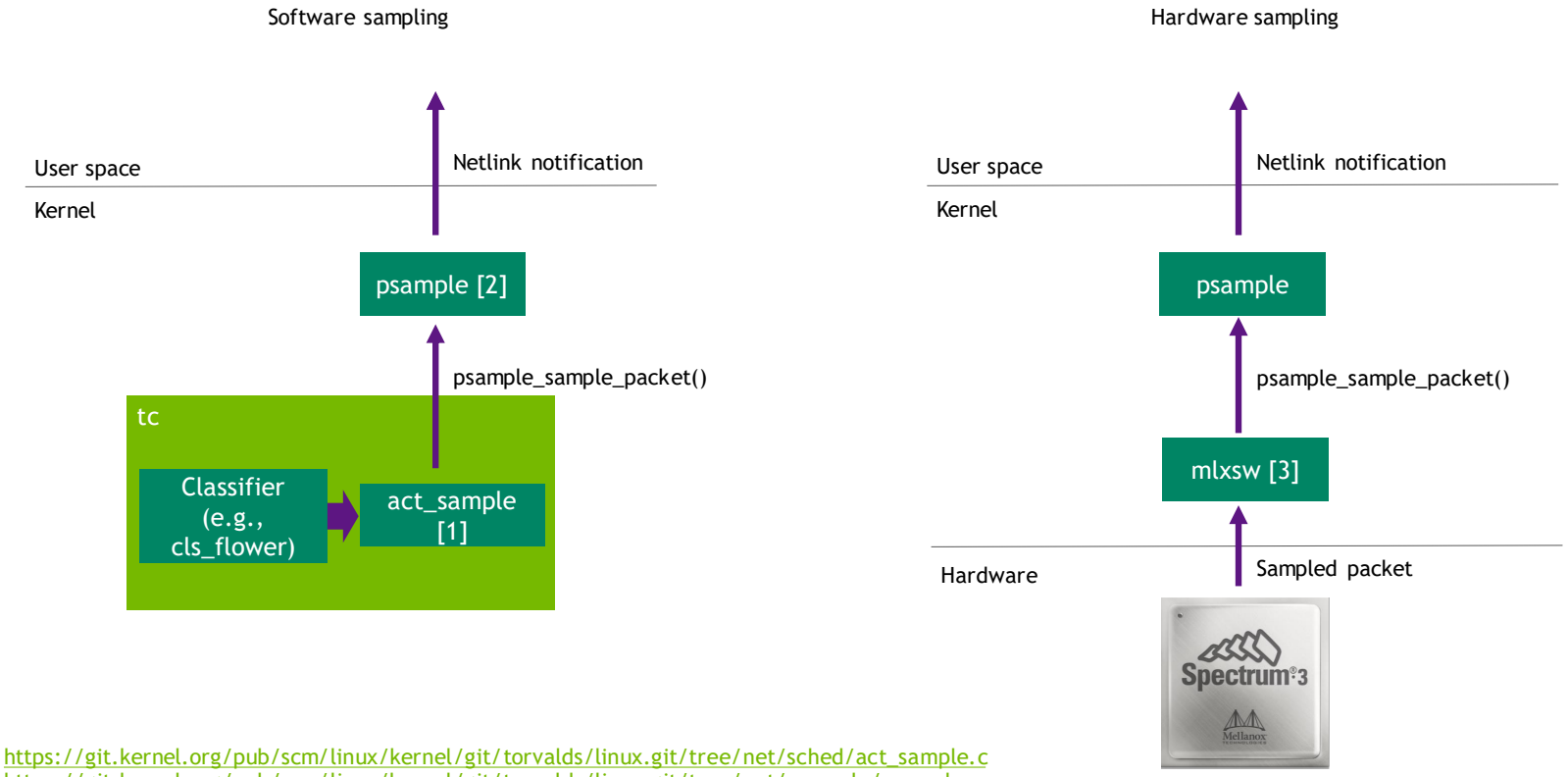
PACKET SAMPLING

- tcpdump(8) only shows control / exception packets on front panel ports
- Sending all Rx-ed / Tx-ed packets to the CPU is sub-optimal
- Hardware sampling / filtering on the other hand:
  - Does not impact data plane operation and performance
  - Provides accurate representation of data plane traffic
  - Has minimal kernel overhead
  - Provides additional metadata [1]

[1] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=2117fce81f6b862aac0673abe8df0c60dca64bfa

NVIDIA.

SAMPLING IN THE KERNEL



[1] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/net/sched/act_sample.c
[2] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/net/psample/psample.c
[3] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/net/ethernet/mellanox/mlxsw

NVIDIA.

USER INTERFACE – CONTROL PLANE

- **Add qdisc**
  ```
  tc qdisc add dev swp1 clsact
  ```
- **Ingress sampling (4.11)**
  ```
  tc filter add dev swp1 ingress pref 1 proto all matchall skip_sw action sample group 1 rate 100 continue
  ```
- **Egress sampling (5.13)**
  ```
  tc filter add dev swp1 egress pref 10 proto all matchall skip_sw action sample group 2 rate 200 continue
  ```
- **Policy-based sampling (5.13)**
  ```
  tc filter add dev swp2 ingress pref 1 proto ip flower skip_sw dst_ip 198.51.100.1 action sample group 3 rate 300

  tc filter add dev swp2 egress pref 10 proto ip flower skip_sw dst_ip 198.51.100.2 action sample group 3 rate 300
  ```


NVIDIA.

USER INTERFACE – DATA PLANE

- Sampled packets are useful for two main purposes
  - Ad-hoc debug of a specific problem on a specific machine by kernel engineers
  - Monitoring of network fabrics / large fleet of machines
- Each purpose is serviced by different tools
  - Ad-hoc debug (CLI): `psample`, `tshark`
  - Monitoring: `hsflowd`
- Multiple tools can consume the information simultaneously

NVIDIA.

USER INTERFACE – DATA PLANE - CLI

- The `psample` [1] utility can be used to display sampled packets' metadata

```
# psample
group 1 in-ifindex 18 out-ifindex 14 origsize 98 sample-rate 100 seq 0 out-tc 0 out-tc-occ 0
latency 512 timestamp Wed Mar 17 22:18:17 2021 454105143 nsec protocol 0x800
```

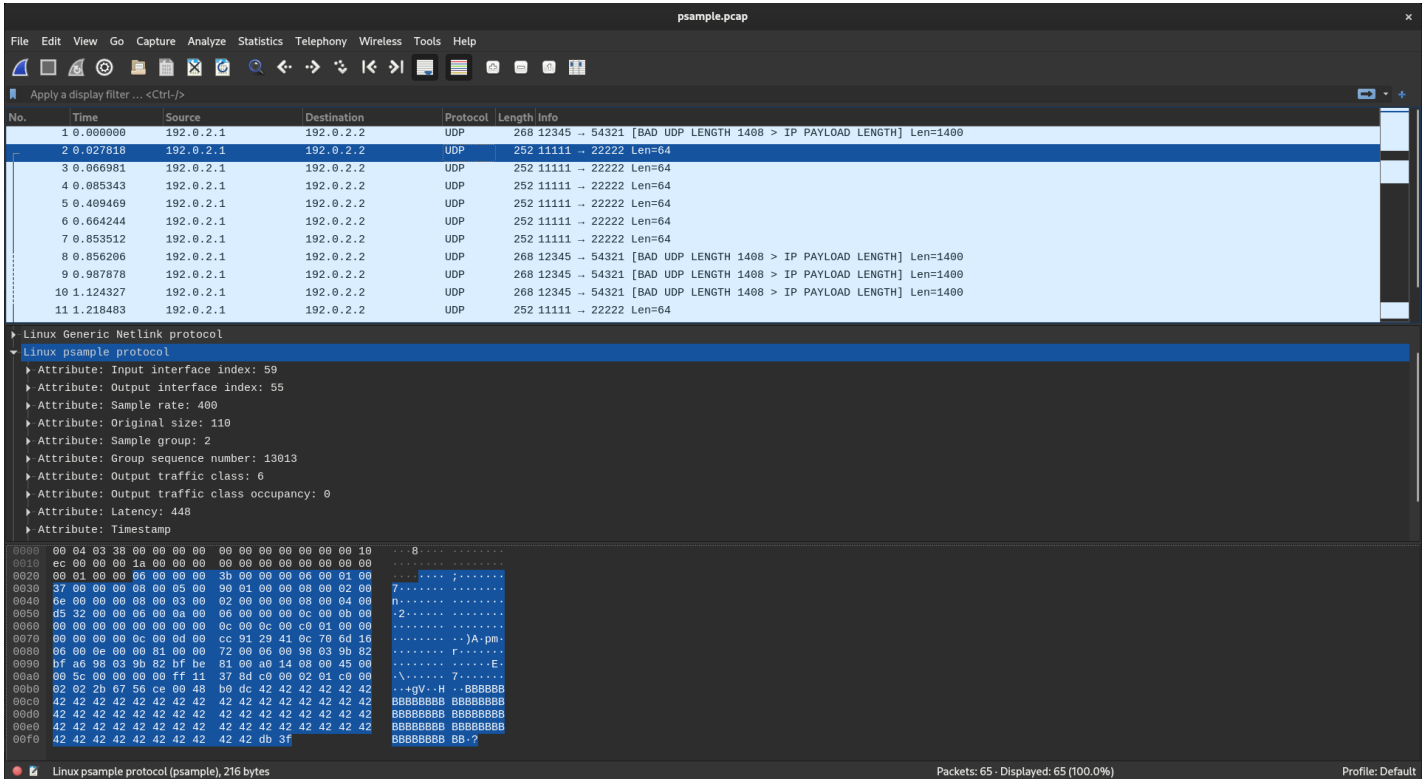
- Psample netlink packets can be dissected by the psample Wireshark dissector [2]

```
# psample -w - | tshark -r - -V
```

[1] https://github.com/Mellanox/libpsample
[2] https://gitlab.com/wireshark/wireshark/-/commit/146578889e18da2f7b5d5dd680c5038bc87e2453

NVIDIA.

USER INTERFACE – DATA PLANE – WIRESHARK



USER INTERFACE – DATA PLANE – TSHARK

```
Linux psample protocol
    Attribute: Input interface index: 59
        Len: 6
        Type: 0x0000, Input interface index (0)
            0... .... .... .... = Nested: False
            .0.. .... .... .... = Network byte order: False
            ..00 0000 0000 0000 = Attribute type: Input interface index (0)
        Input interface index: 0x003b
    Attribute: Output interface index: 55
        Len: 6
        Type: 0x0001, Output interface index (1)
            0... .... .... .... = Nested: False
            .0.. .... .... .... = Network byte order: False
            ..00 0000 0000 0001 = Attribute type: Output interface index (1)
        Output interface index: 0x0037
    Attribute: Sample rate: 400
        Len: 8
        Type: 0x0005, Sample rate (5)
            0... .... .... .... = Nested: False
            .0.. .... .... .... = Network byte order: False
            ..00 0000 0000 0101 = Attribute type: Sample rate (5)
        Sample rate: 400
    Attribute: Original size: 1446
        Len: 8
        Type: 0x0002, Original size (2)
            0... .... .... .... = Nested: False
            .0.. .... .... .... = Network byte order: False
            ..00 0000 0000 0010 = Attribute type: Original size (2)
        Original size: 0x000005a6
    Attribute: Sample group: 1
        Len: 8
        Type: 0x0003, Sample group (3)
            0... .... .... .... = Nested: False
            .0.. .... .... .... = Network byte order: False
            ..00 0000 0000 0011 = Attribute type: Sample group (3)
        Sample group: 1
```


NVIDIA.

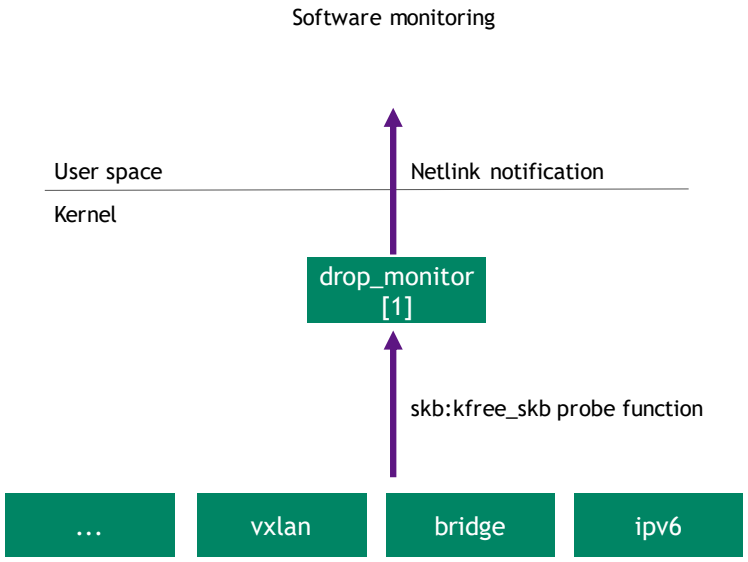
PACKET DROPS MONITORING

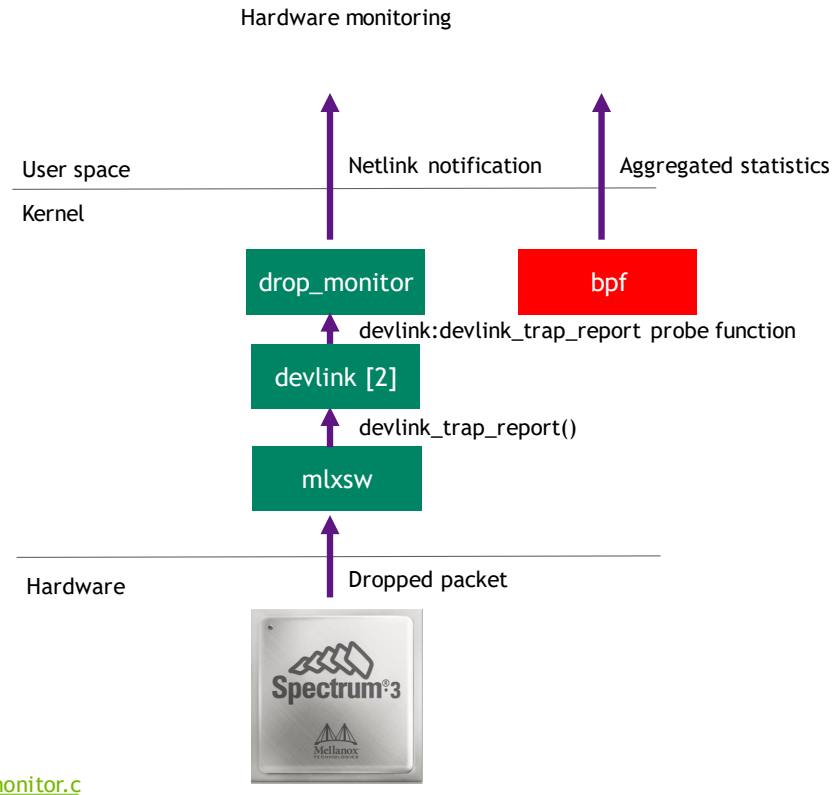
- Packet drops in the kernel can be traced using `skb:kfree_skb` tracepoint
- The `drop_monitor` kernel module registers its probe on the tracepoint and emits netlink notifications
- Since kernel 5.4 [1], drop_monitor can encode truncated packets and metadata in the notification
- Packets in the offloaded data plane can also be dropped
- Switch ASIC can be programmed to trap such packets to the CPU for monitoring purposes

[1] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=ca30707dee2bc8bc81cfd8b4277fe90f7ca6df1f

NVIDIA.

MONITORING PACKET DROPS IN THE KERNEL





[1] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/net/core/drop_monitor.c
[2] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/net/core/devlink.c

NVIDIA.

USER INTERFACE – CONTROL PLANE

- **Enable / disable a packet trap (5.4)**
  ```
  devlink trap set pci/0000:01:00.0 trap blackhole_route action trap
  devlink trap set pci/0000:01:00.0 trap blackhole_route action drop
  ```

- **Police packet traps (5.7)**
  ```
  devlink trap policer set pci/0000:01:00.0 policer 1 rate 1024 burst 256
  ```


NVIDIA.

USER INTERFACE – DATA PLANE

- Like sampled packets, dropped packets are useful for two main purposes
  - Ad-hoc debug of a specific problem on a specific machine by kernel engineers
  - Monitoring of network fabrics / large fleet of machines
- Each purpose is serviced by different tools
  - Ad-hoc debug (CLI): `dropwatch`, `dwdump`, `tshark`
  - Monitoring: `hsflowd`, `prometheus`
- Multiple tools can consume the information simultaneously

NVIDIA.

USER INTERFACE – DATA PLANE - CLI

- The `dropwatch` [1] utility can be used to display dropped packets' metadata

```
dropwatch> start
Enabling monitoring...
Kernel monitoring activated.
Issue Ctrl-C to stop monitoring
drop at: ingress_vlan_filter (l2_drops)
origin: hardware
input port ifindex: 9
input port name: swp3
timestamp: Mon Aug 26 20:15:33 2019 461798287 nsec
protocol: 0x8100
length: 110
original length: 110
```

- Drop monitor netlink packets can be dissected by the drop monitor Wireshark dissector [2]

```
# dwdump –o hw -w - | tshark -r - -V
```

[1] https://github.com/nhorman/dropwatch
[2] https://gitlab.com/wireshark/wireshark/-/commit/a94a860c0644ec3b8a129fd243674a2e376ce1c8

NVIDIA.

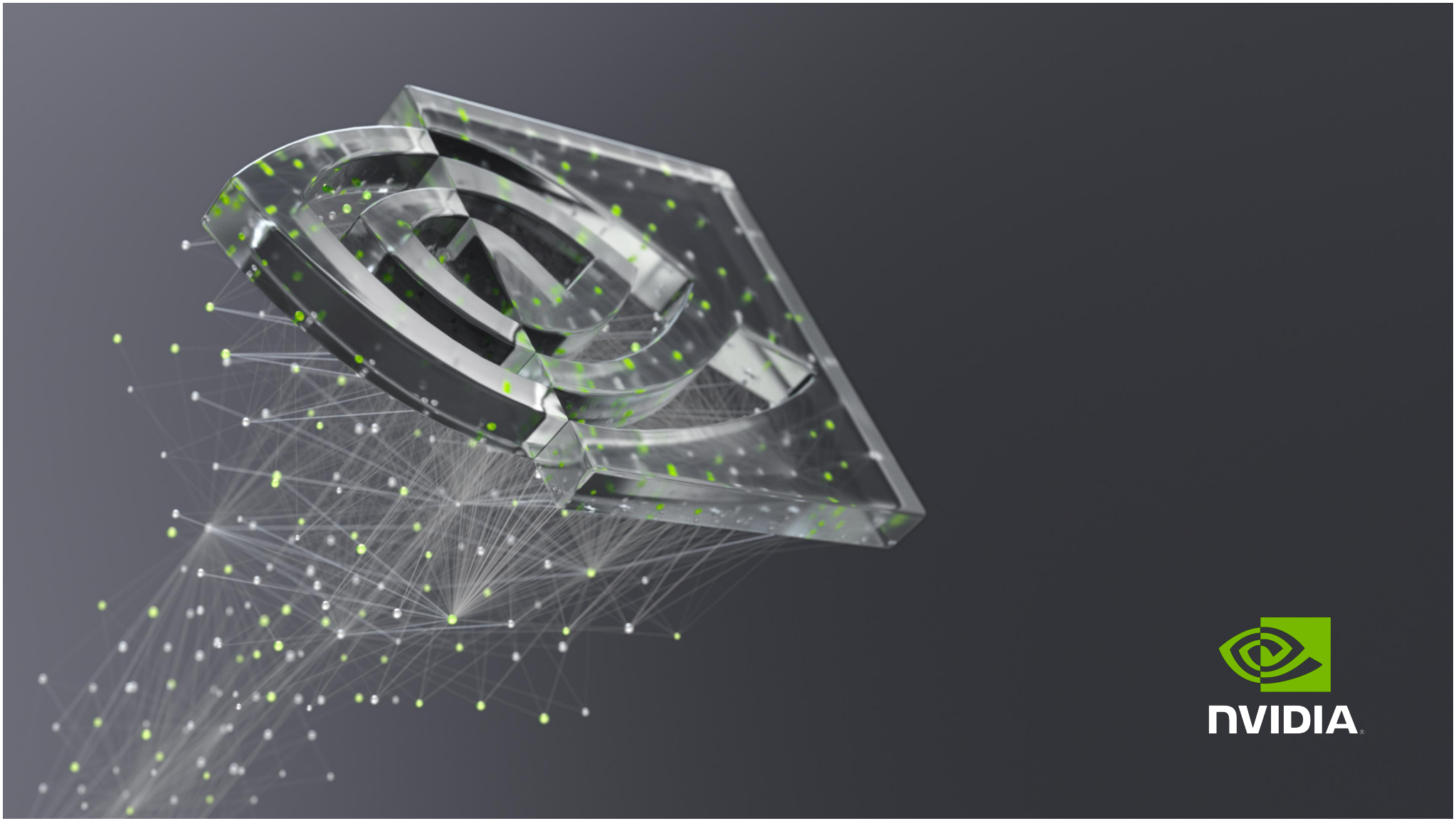
USER INTERFACE – DATA PLANE - BPF

- A BPF program [1] can be attached to the devlink:devlink_trap_report tracepoint [2] in order to get aggregated per-{trap, flow} statistics
- The BPF map storing the aggregated statistics can be periodically exported to Prometheus and visualized using Grafana:



[1] https://github.com/Mellanox/mlxsw/blob/master/Debugging/libbpf-tools/src/trapagg_example.txt
[2] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=f2e834694b0d92187d889172da842e27829df371

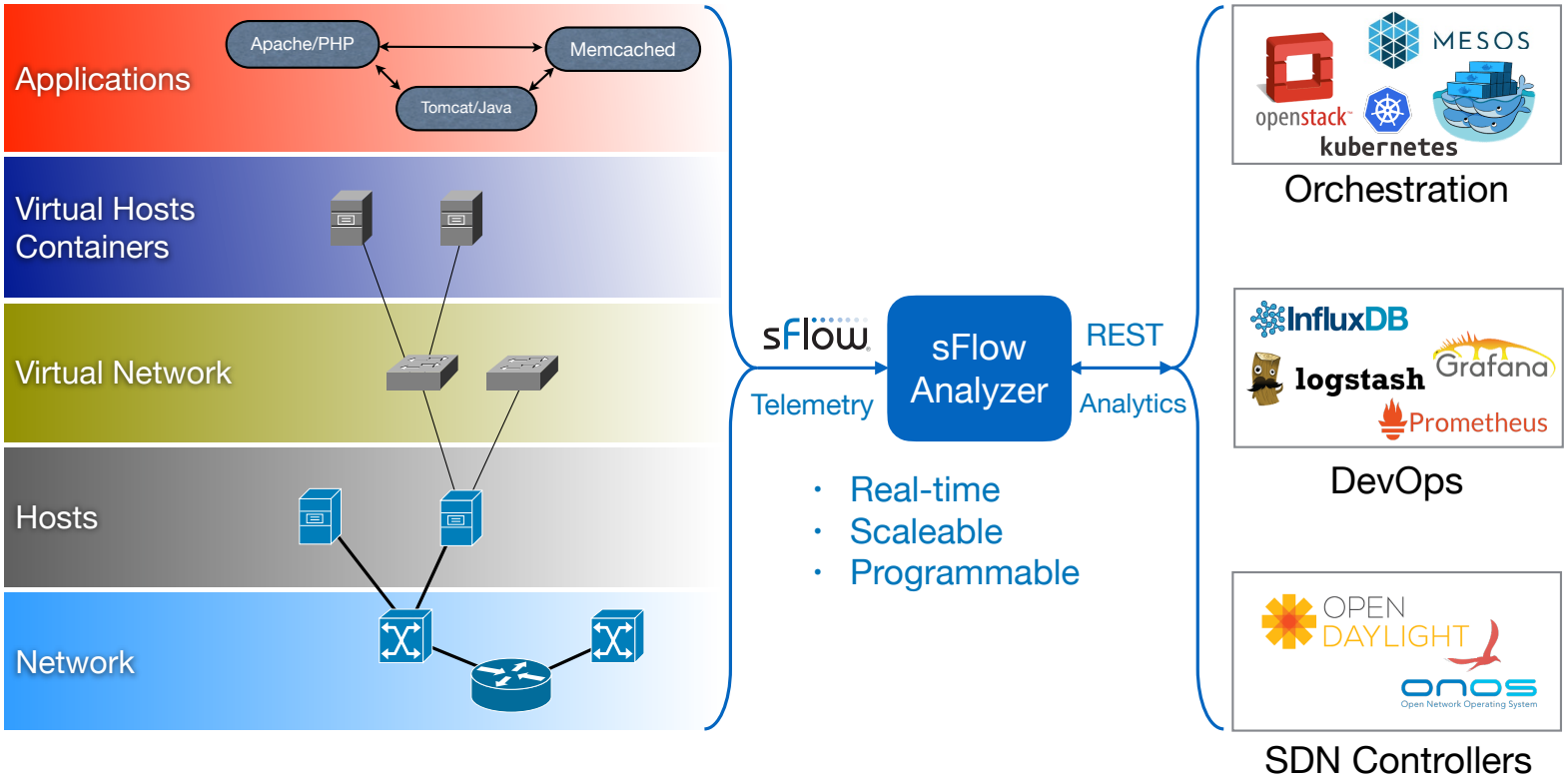
NVIDIA.


NVIDIA

Linux sFlow[1]

1. sFlow.org

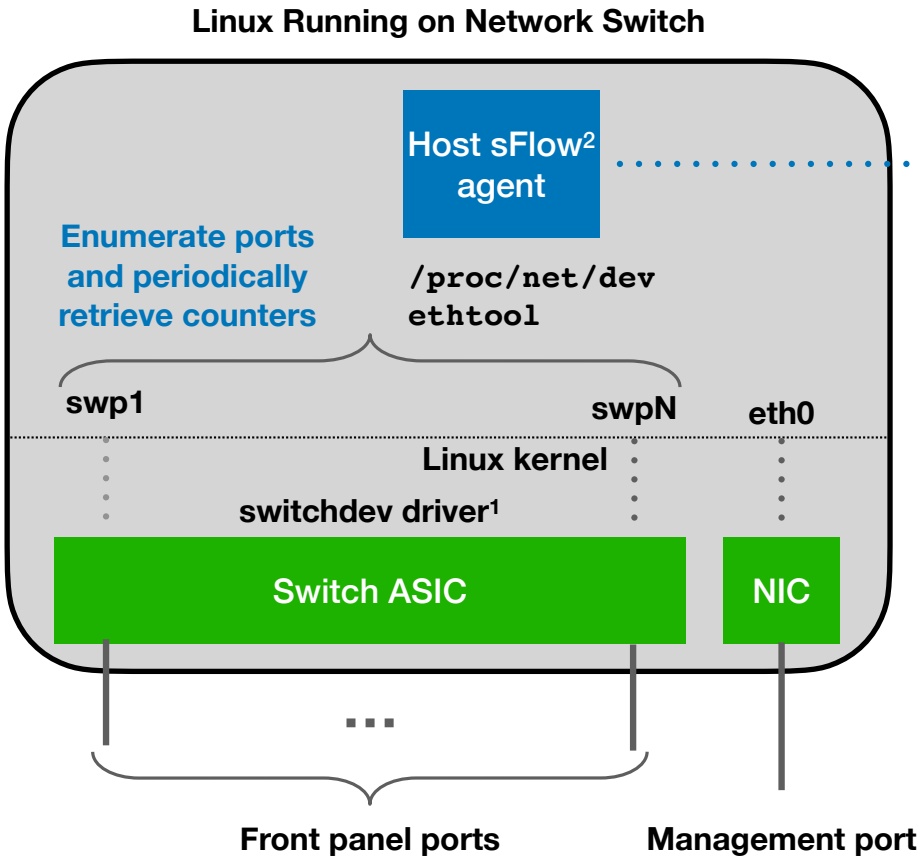
sFlow system architecture



Embedded monitoring of all switches, all routers, all servers, all applications, all the time

Consistent measurements shared between multiple management tools

Linux: Streaming counters

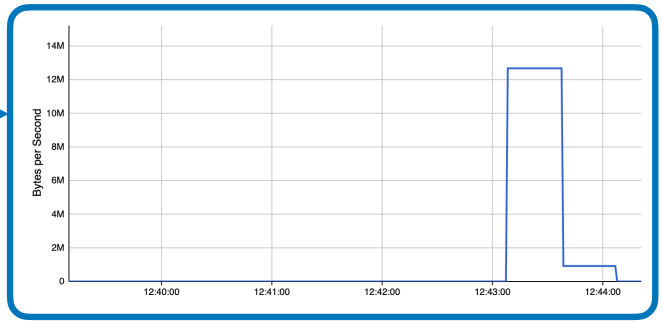


sFlow

XDR[3] records / UDP

- switch port counters[4]
- SFP/QSFP optics metrics[5]
- cpu, memory, disk[6]
- systemd service metrics[6]

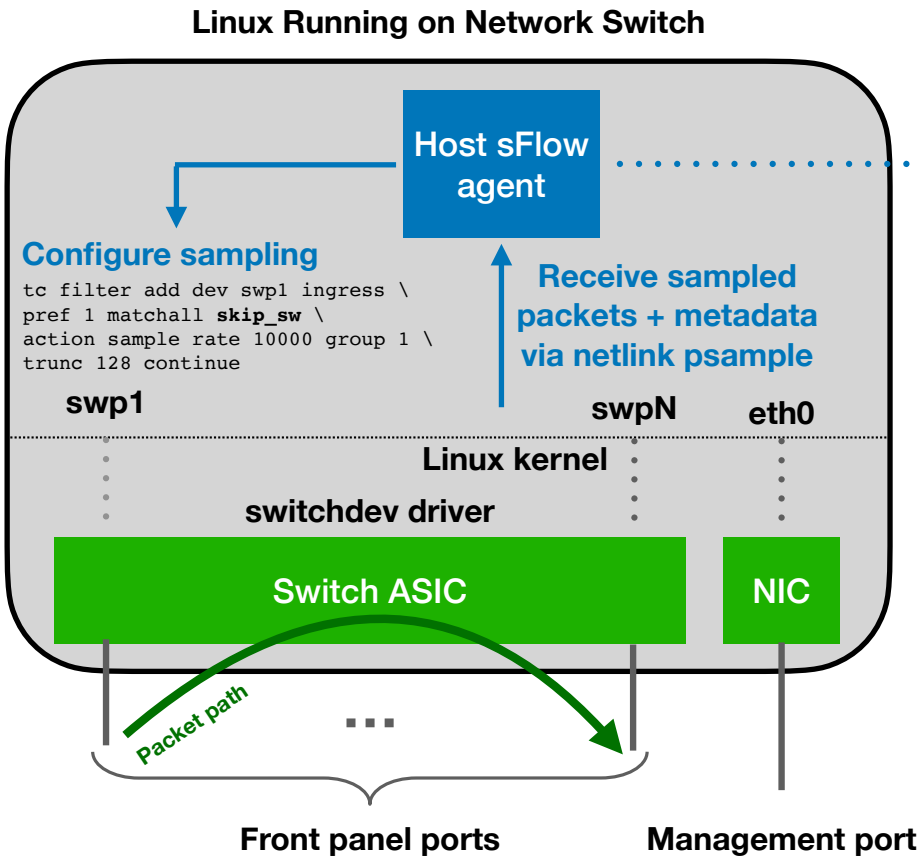
sFlow analyzer[7]



- Port counters summarize packets to show spike in traffic
- Need to break out details to identify type and source of traffic
- **How do you get details when ASIC handles billions of packets per second?**

1. https://www.kernel.org/doc/html/latest/networking/switchdev.html
2. https://github.com/sflow/host-sflow
3. XDR: External Data Representation Standard, RFC 4506
4. https://sflow.org/sflow_version_5.txt
5. https://sflow.org/sflow_optics.txt
6. https://sflow.org/sflow_host.txt
7. https://github.com/sflow-rt/browse-metrics

Linux: Streaming sampled packets

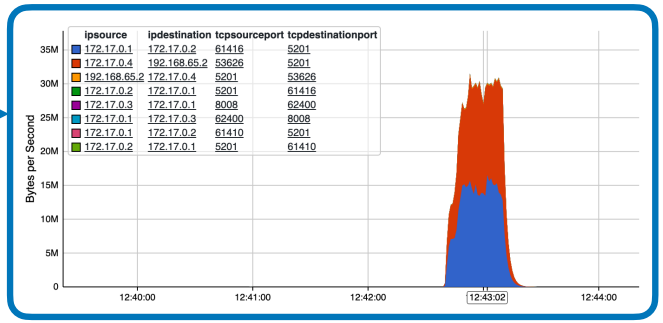


sFlow

XDR records / UDP

- packet header[1]
- frame length[1]
- sampling rate[1]
- ingress / egress port[1]
- egress queue[2]
- egress queue depth[2]
- nanosecond transit delay[2]

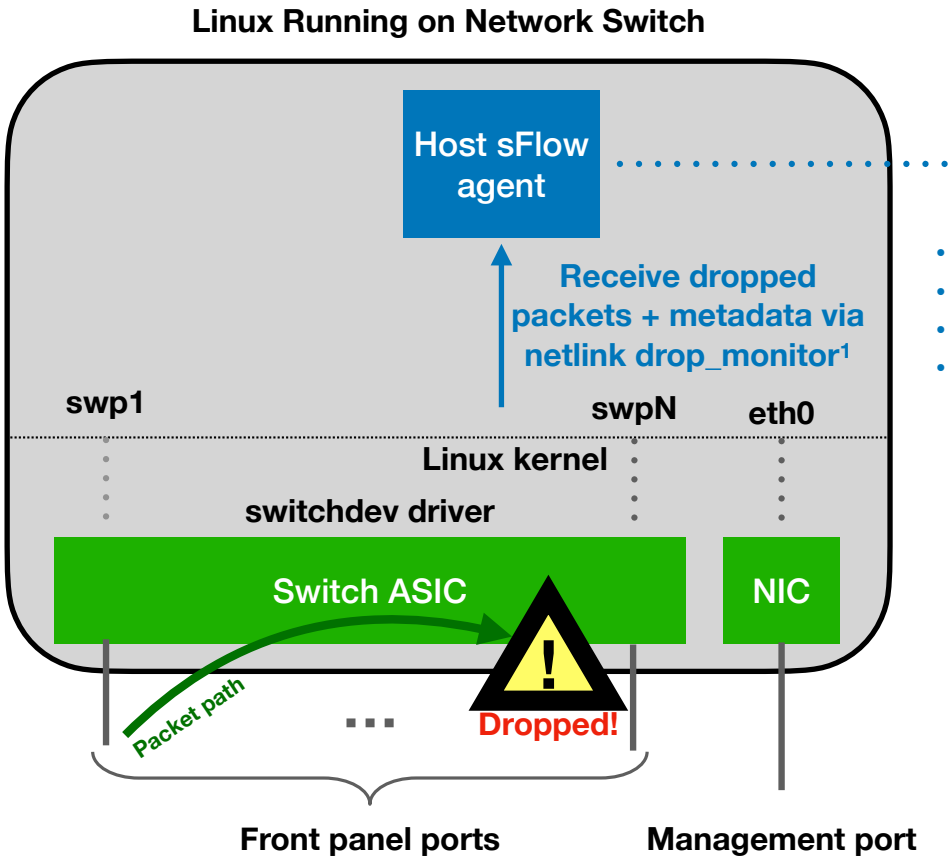
sFlow analyzer[3]



- Packet sampling provides visibility into packet flows
- Identify source/destination addresses and ports
- Sub-second detection of large "Elephant" flows
- **What about failed connections?**

1. https://sflow.org/sflow_version_5.txt
2. https://sflow.org/sflow_transit.txt
3. https://github.com/sflow-rt/browse-flows

Linux: Streaming dropped packets

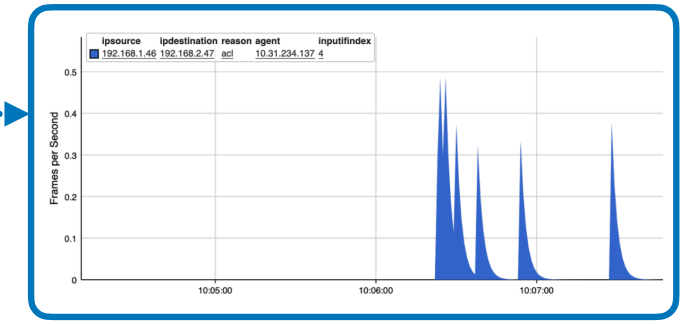


sFlow

XDR records / UDP

- packet header[2]
- frame length[2]
- ingress port[2]
- drop reason[2]

sFlow analyzer[3]



- Network-wide visibility into each dropped packet, even if it is a 1-in-a-billion event
- Dropped packets can severely impact performance and availability of critical services
- Identify sources, destinations, protocols, locations of discards
- Troubleshoot firewalls, microbursts, routing loops, black hole route, MTU etc.

1. https://www.kernel.org/doc/html/latest/networking/devlink/devlink-trap.html
2. https://sflow.org/sflow_drops.txt
3. https://github.com/sflow-rt/browse-drops

Host sFlow agent default settings

Configure Host sFlow agents

Edit `/etc/hsflowd.conf`

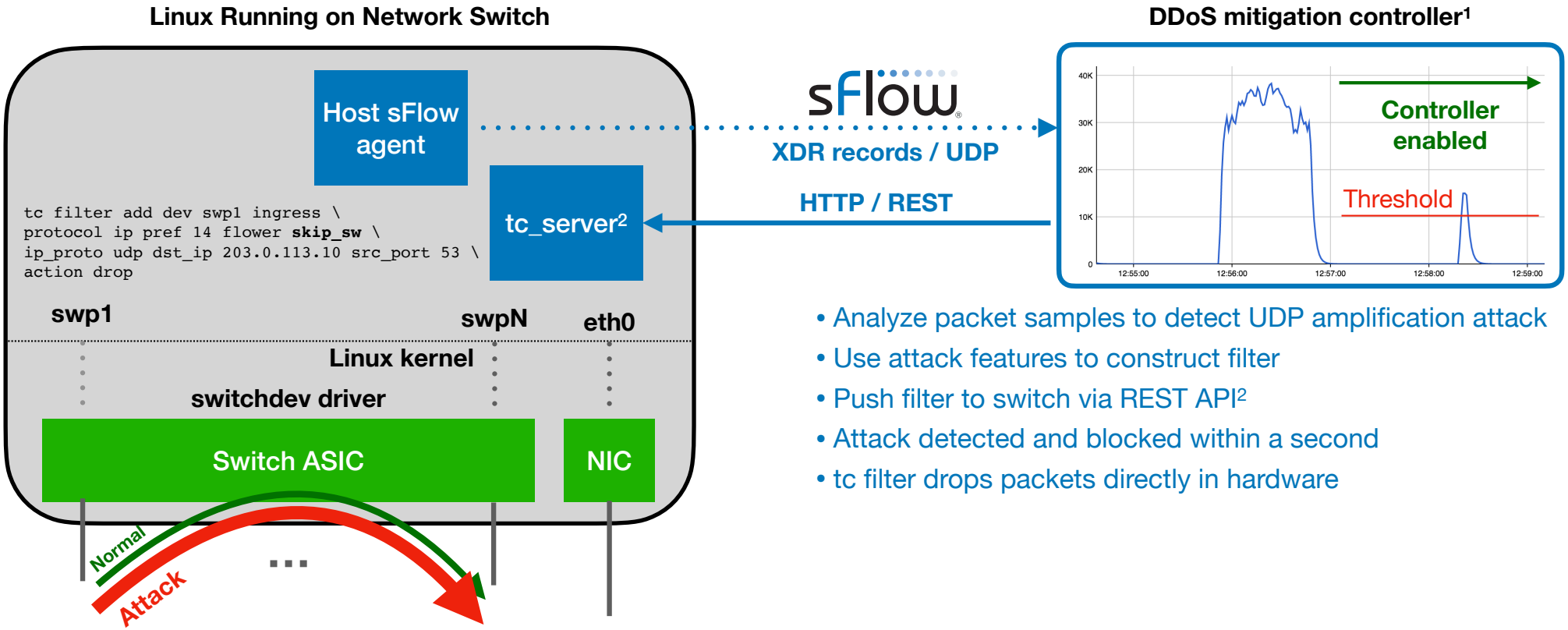
```
sflow {
  collector { ip=172.20.20.1 }
  systemd { }
  psample { group=1 egress=on }
  dropmon { group=1 start=on sw=off hw=on }
  dent { sw=off switchport=swp.* }
}
```

Same configuration on all switches reduces operational complexity

Link Speed	Samping Rate	Polling Interval
1G	1-in-1,000	30 seconds
2.5G	1-in-2,500	30 seconds
5G	1-in-5,000	30 seconds
10G	1-in-10,000	30 seconds
25G	1-in-25,000	30 seconds
40G	1-in-40,000	30 seconds
50G	1-in-50,000	30 seconds
100G	1-in-100,000	30 seconds
400G	1-in-400,000	30 seconds

The default settings ensure that large flows (defined as consuming 10% of link bandwidth) are detected within approximately 1 second. Counter polling and packet sampling are enabled on every port on every device for data center wide visibility.

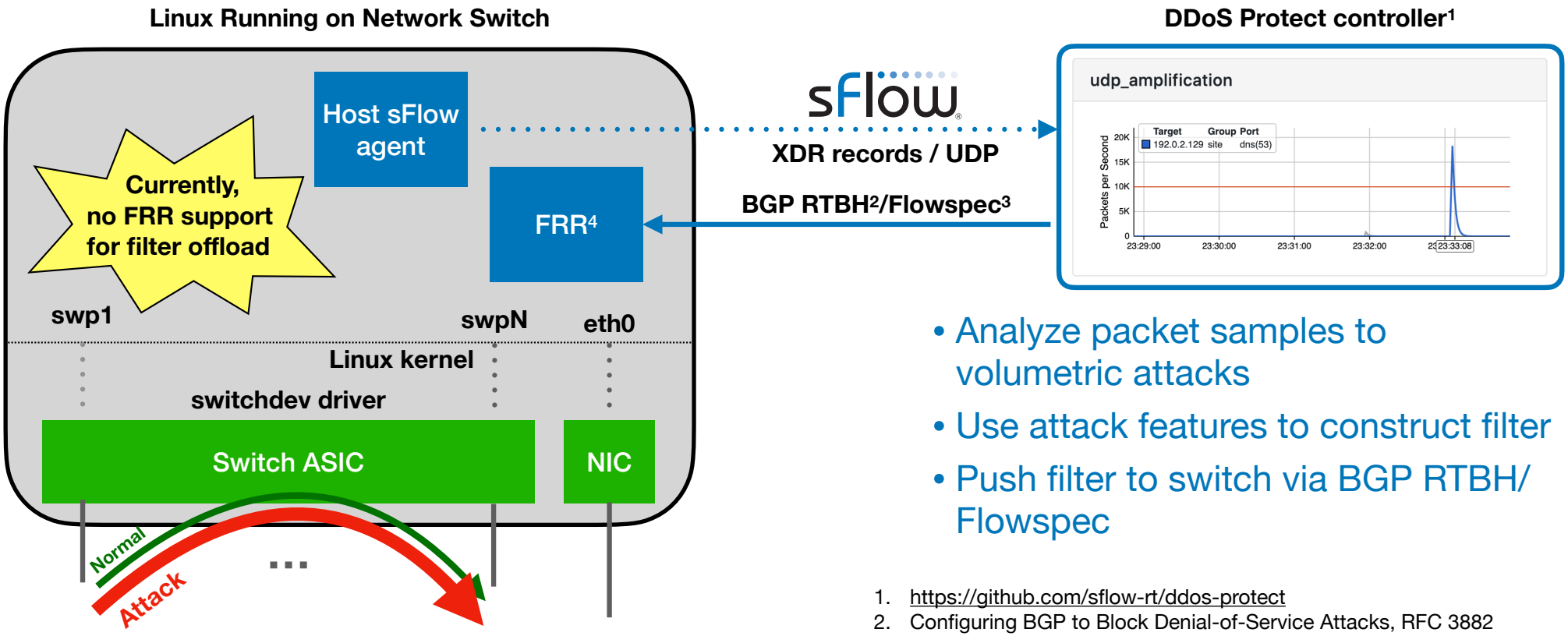
DDoS mitigation example



- Analyze packet samples to detect UDP amplification attack
- Use attack features to construct filter
- Push filter to switch via REST API[2]
- Attack detected and blocked within a second
- tc filter drops packets directly in hardware

1. https://blog.sflow.com/2021/06/ddos-mitigation-using-linux-switch.html
2. https://github.com/sflow-rt/tc_server

DDoS mitigation with BGP



- Analyze packet samples to volumetric attacks
- Use attack features to construct filter
- Push filter to switch via BGP RTBH/ Flowspec

1. https://github.com/sflow-rt/ddos-protect
2. Configuring BGP to Block Denial-of-Service Attacks, RFC 3882
3. Dissemination of Flow Specification Rules, RFC 8955
4. https://frrouting.org/

Benefits of Linux networking

- Freedom to run any Linux distribution on switch
- Use standard Linux tools, monitoring, and automation solutions
- Reduce complexity (and failures) by eliminating unnecessary services
- Easy for developers to customize switches by adding measurement and control agents using standard Linux APIs
- Develop and test software on VMs / containers before pushing into production
- Large developer community (e.g. Stack Overflow, GitHub etc.)